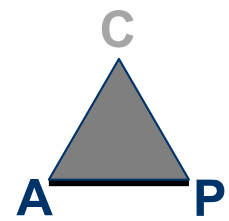
CompSci 390-03-26S

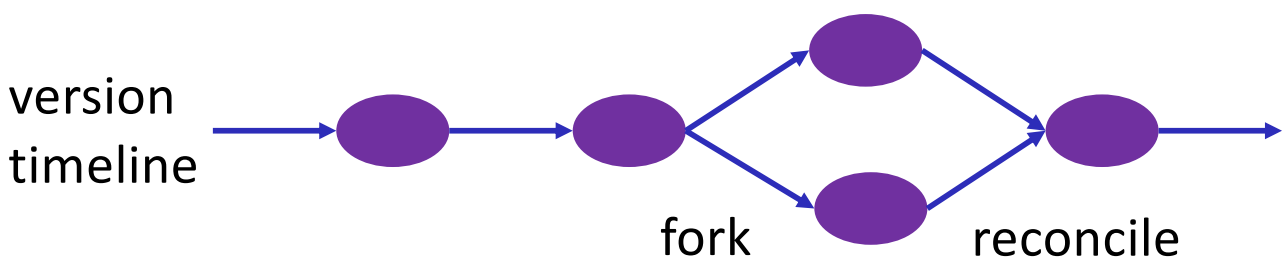
Distributed and Networked Systems

Logical Clocks and Concurrency

AP: Availability, but at what cost?

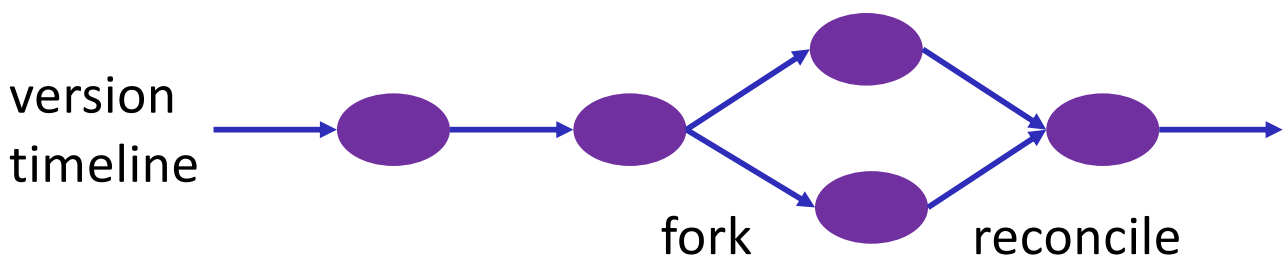
- **How can we build systems that work** when network connectivity is varying/intermittent? **AP "always on"**
- **CAP/FLP** says: give up on atomic consistency. ~~C~~
- What are we left with? Anything usable?
  - **Eventual** consistency: weak!
  - **Causal consistency**: better!
- How to resolve conflicts and merge/reconcile forks?
  - Can we hide this mess from applications?





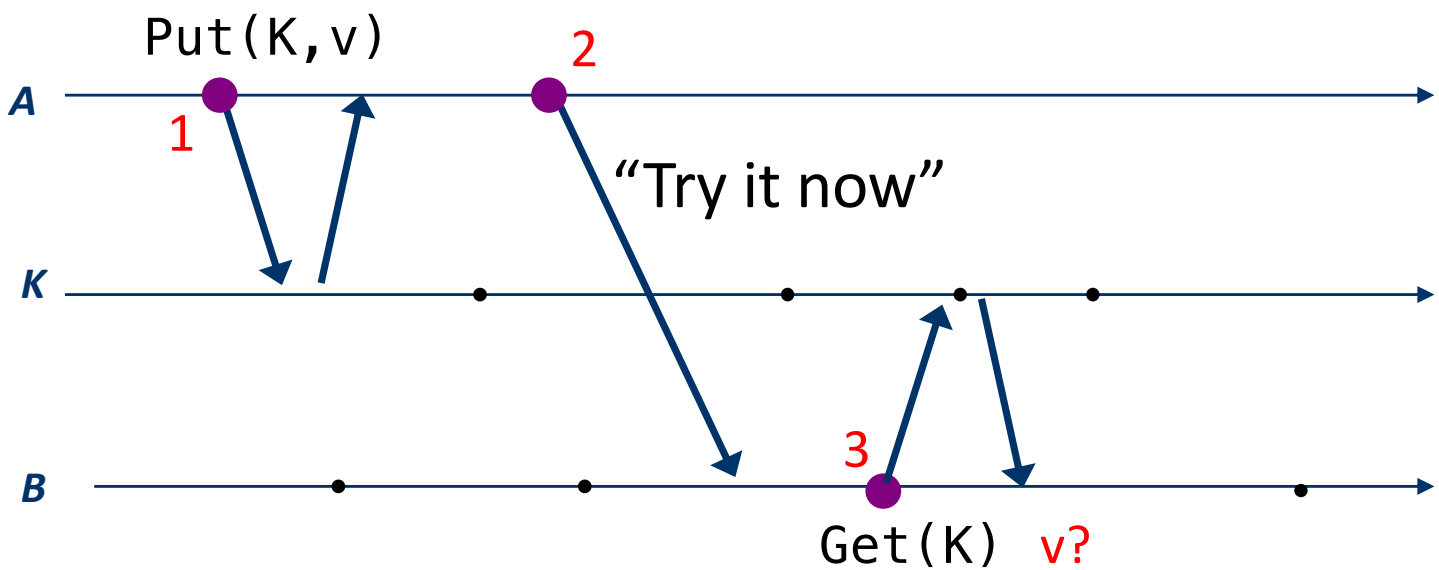
Write conflicts and timeline forks

- In addition to **stale reads**, eventual consistency (and other AP systems) must handle **write conflicts**, which cause a "fork in the timeline".
  - Forks never happen with CP consensus (e.g., Raft): a strong primary/leader imposes mutual exclusion to order conflicting writes.
  - AP systems (e.g., Dynamo) allow **concurrent** writes and **concurrent** read/write to a key K. **Racy**! But AP demands it (why?).
  - Write conflicts result in multiple values/versions for a key → fork.



"Try it now"

- Atomic consistency (linearizability) behaves as expected in real time, e.g., for communication external to the "system". **("External consistency")**
- Eventual consistency with stale reads might not...
- This violates our notion of **causality.** 1→2→3



Logical time and causality

- **Premise**. Nodes communicate only by "touchpoints", e.g., messages: they can observe a remote event only through some chain of messages.
- **Causality**. Event e_1 can affect or cause event e_2 iff the node initiating e_2 could have already observed e_1.
  - Message patterns define a **potential** causality relation.
  - Also known as happened-before (→).
- **Logical time**. Event e_1 precedes e_2 iff e_1 could have caused e_2.
  - We say $e_1 \rightarrow e_2$ or $e_1 < e_2$.
- **Concurrency**. Events are concurrent iff neither precedes the other.
  - Happened-before is a **partial order**.

Causal consistency

- Causality defines a **partial order** of events/updates, which we define formally as Lamport's *happened-before* relation. →
- Causality captures a key aspect of what we mean by consistency.
- An important class of systems provide **causal consistency**, which preserves causal orderings: good enough?
  - Multicore memory systems
  - Always preserve causal orders induced by synchronization (e.g., locks)
  - i.e., a release→acquire pair (lock handoff) means "try it now".
- Causal consistency is **"weak enough" to admit** AP, if the application can reconcile forks (write conflict resolution, partition recovery).

Consistency, Availability, and Convergence

Prince Mahajan, Lorenzo Alvisi, and Mike Dahlin

Department of Computer Science, The University of Texas at Austin

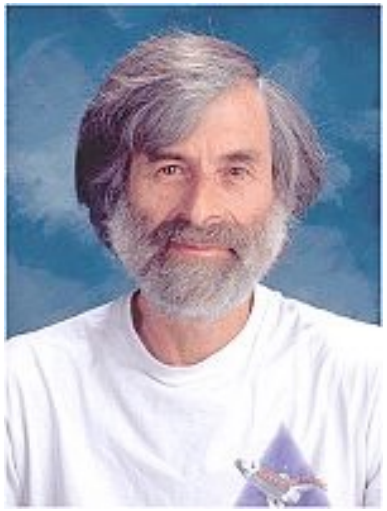
Technical Report (UTCS TR-11-22)

Abstract

We examine the limits of consistency in fault-tolerant distributed storage systems. In particular, we identify fundamental tradeoffs among properties of *consistency*, *availability*, and *convergence*, and we close the gap between what is known to be impossible (i.e. CAP) and known systems that are highly-available but that provide weaker consistency such as causal. Specifically, in the asynchronous model with omission-failures and unreliable networks, we show the following tight bound: **No consistency stronger than Real Time Causal Consistency (RTC) can be provided in an always-available, one-way convergent system** and **RTC can be provided in an always-available, one-way convergent system.** In the asynchronous, Byzantine-failure model, we show that it is impossible to implement many of the recently introduced *fork*-based consistency semantics without sacrificing either availability or convergence; notably, proposed systems allow Byzantine nodes to permanently partition correct nodes from one another. To address this limitation, we introduce *bounded fork join causal semantics* that extends causal consistency to Byzantine environments while retaining availability and convergence.

A Classic Paper

- **Time, Clocks, and the Ordering of Events in Distributed Systems**
- Leslie Lamport, CACM, July 1978
- Introduced:
  - logical clocks ("Lamport clocks")
  - state-machine replication model
  - causal multicast / propagation
  - physical clocks respecting causality
  - (Almost but not) vector clocks



“Space-time diagram”

Fig. 1.

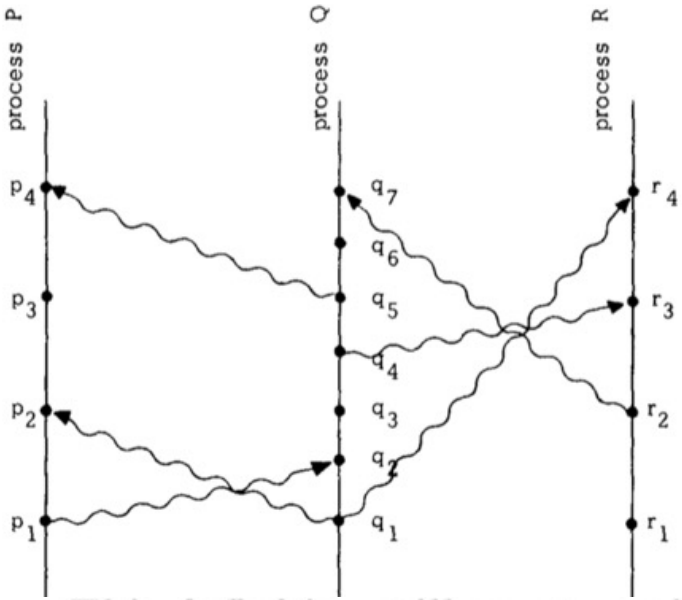
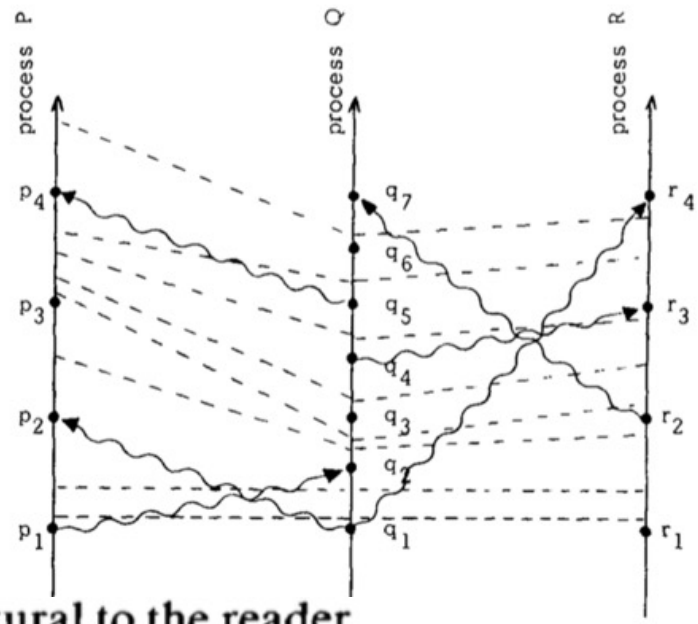
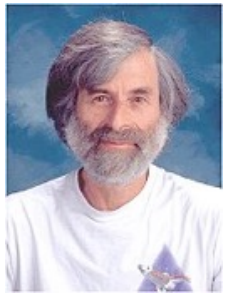


Fig. 2.





This definition will appear quite natural to the reader familiar with the invariant space-time formulation of special relativity, as described for example in [1] or the first chapter of [2]. In relativity, the ordering of events is defined in terms of messages that *could* be sent. However,

DOI:10.1145/2890782

Article development led by acmqueue
queue.acm.org

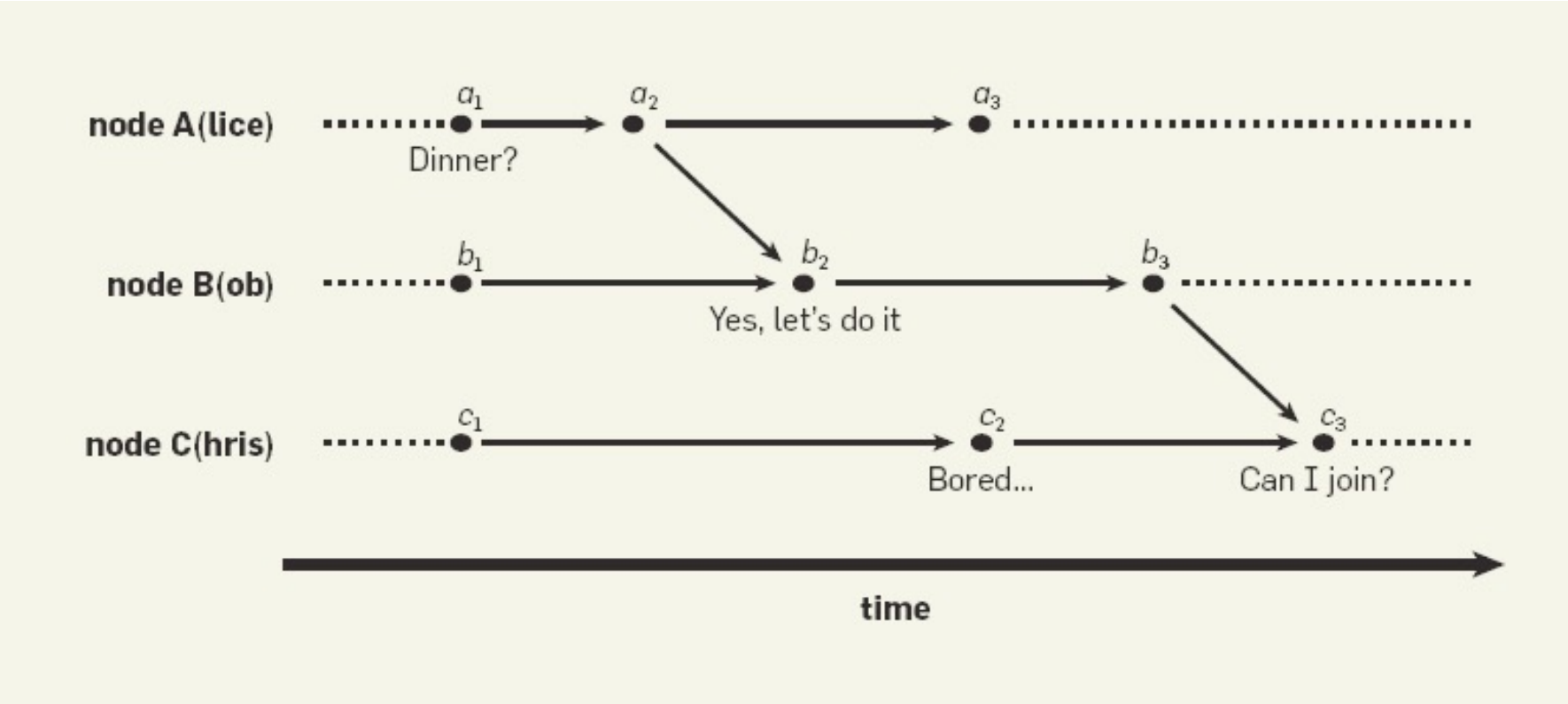
Sometimes all you need is the right language.

BY CARLOS BAQUERO AND NUNO PREGUIÇA

Why Logical Clocks Are Easy

APRIL 2016 VOL. 59 NO. 4
COMMUNICATIONS OF THE ACM

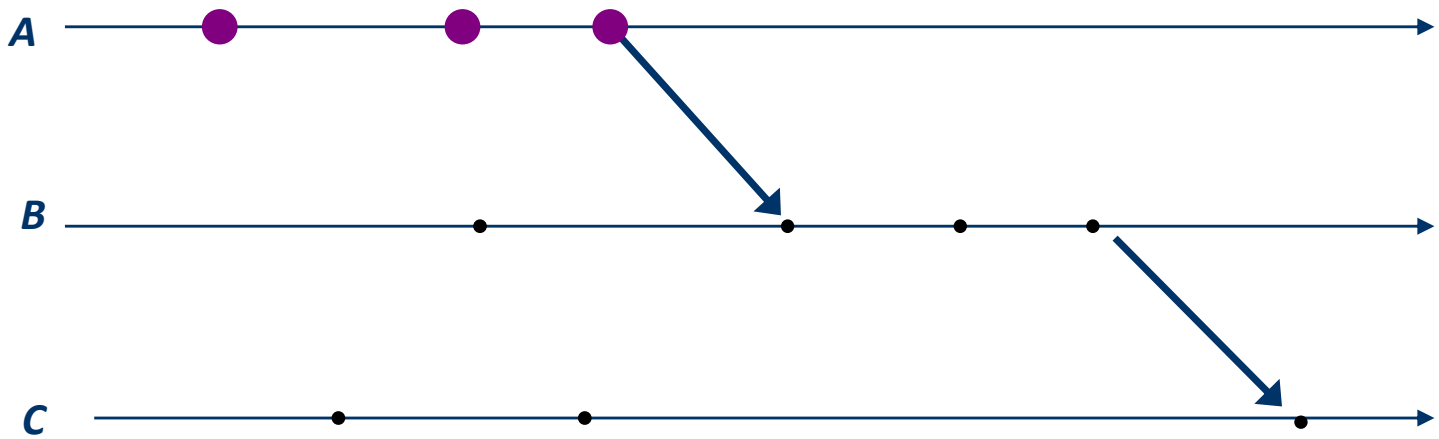
Happened-before

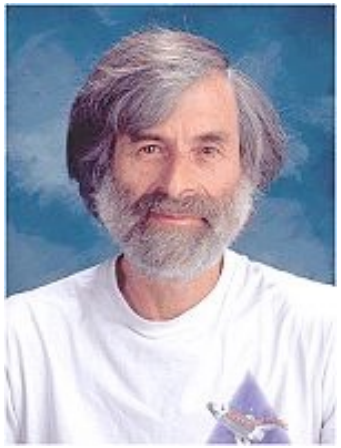


APRIL 2016 VOL. 59 NO. 4 COMMUNICATIONS OF THE ACM

Axiom 1: happened-before (→)

- If *e1, e2* are in the same process/thread, and *e1* comes before *e2*, then *e1*→*e2*.
  - Also called **program order**

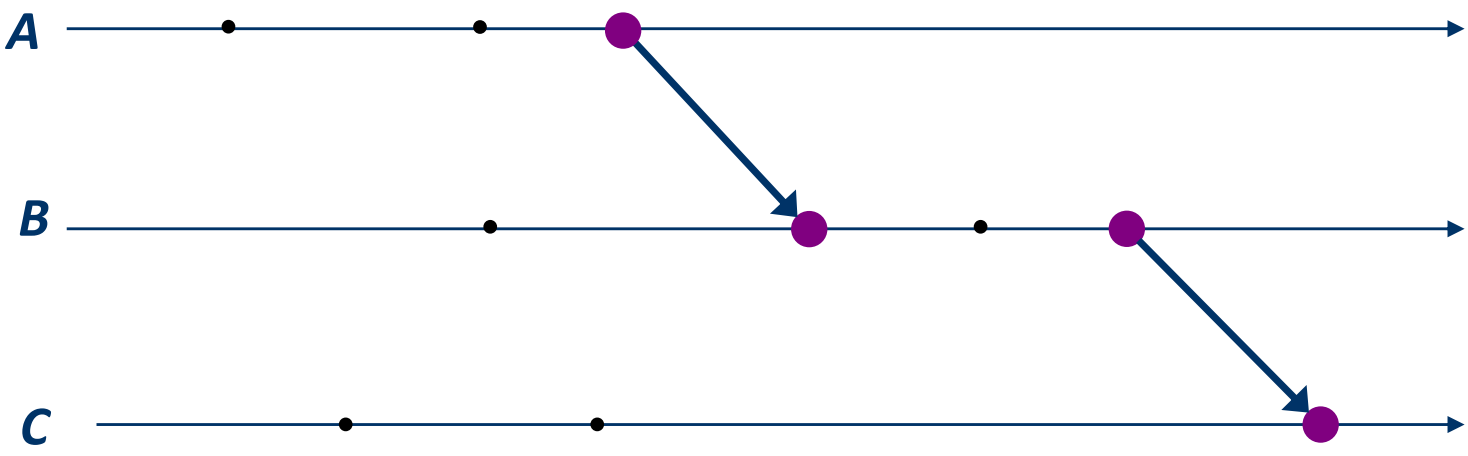


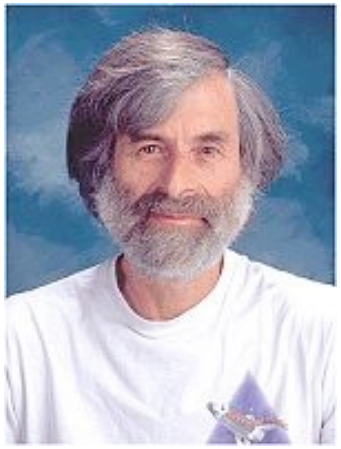


Time, Clocks, and the Ordering of Events in Distributed systems, by Leslie Lamport, CACM 21(7), July 1978

Axiom 2: happened-before (→)

- If e1 is a message send, and e2 is the corresponding receive, then e1→e2. [Also: mutex handoff: release→next acquire.]

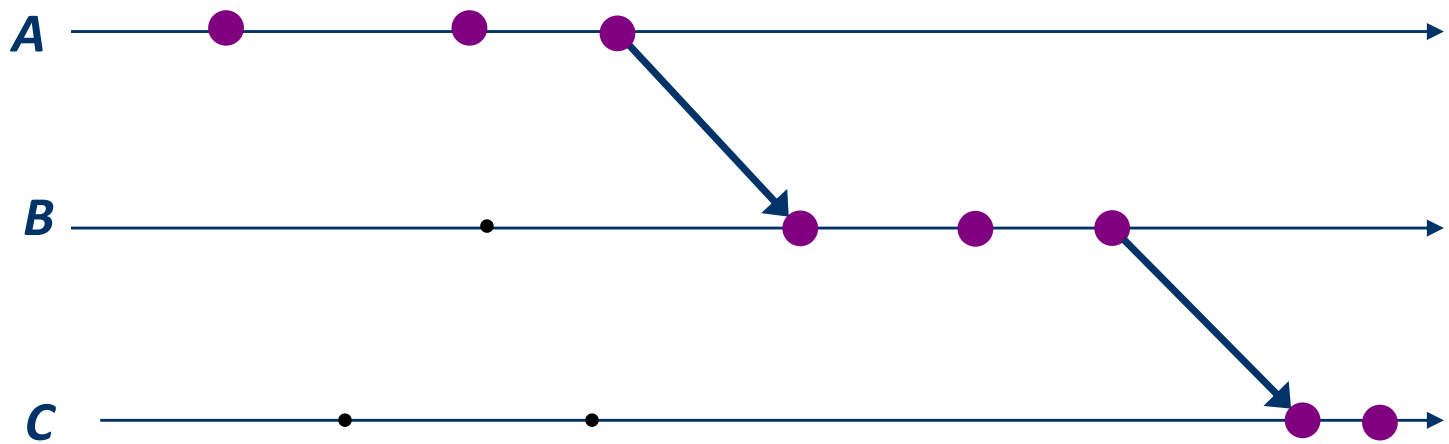


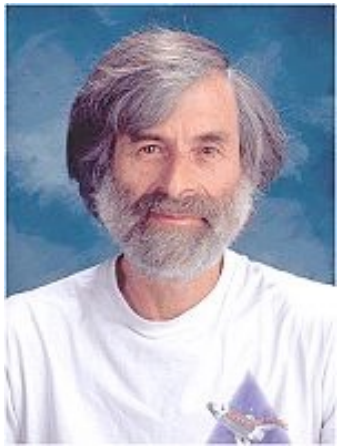


Time, Clocks, and the Ordering of Events in Distributed systems, by Leslie Lamport, CACM 21(7), July 1978

Axiom 3: happened-before (→)

- → is transitive: happened-before is the transitive closure of the relation defined by Axiom 1 and Axiom 2

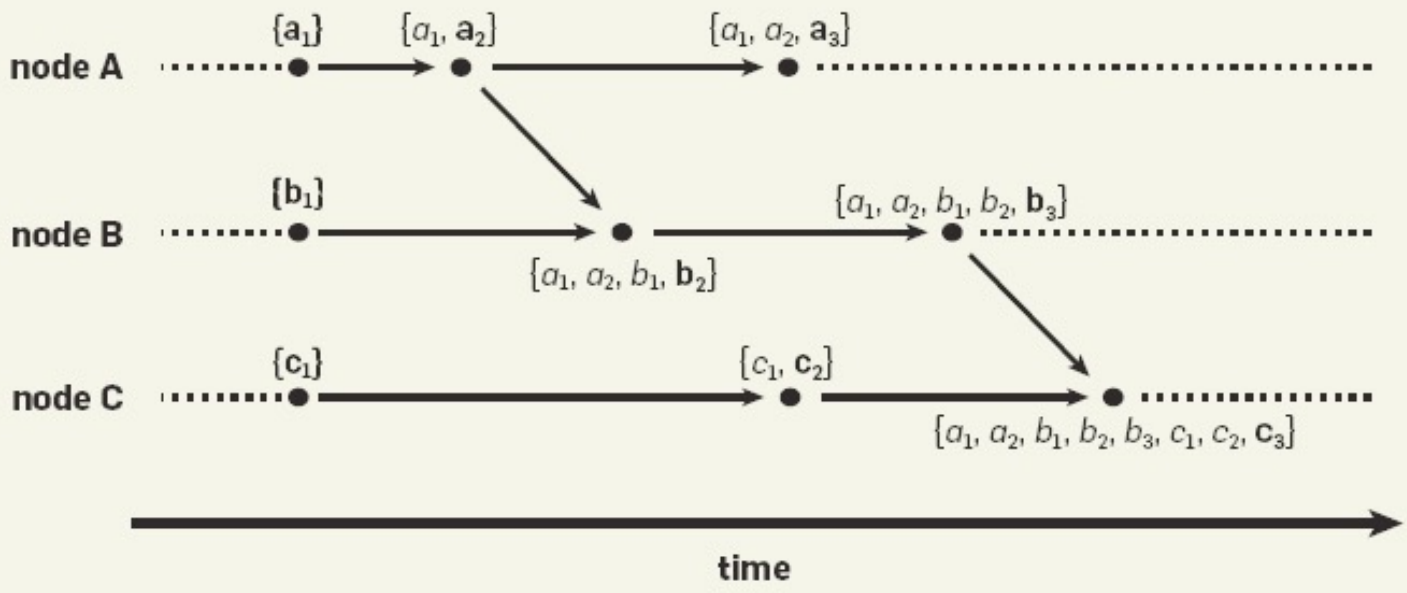




Time, Clocks, and the Ordering of Events in Distributed systems, by Leslie Lamport, CACM 21(7), July 1978

Causal dependencies

Lists the events that **causally precede** each timeline event:



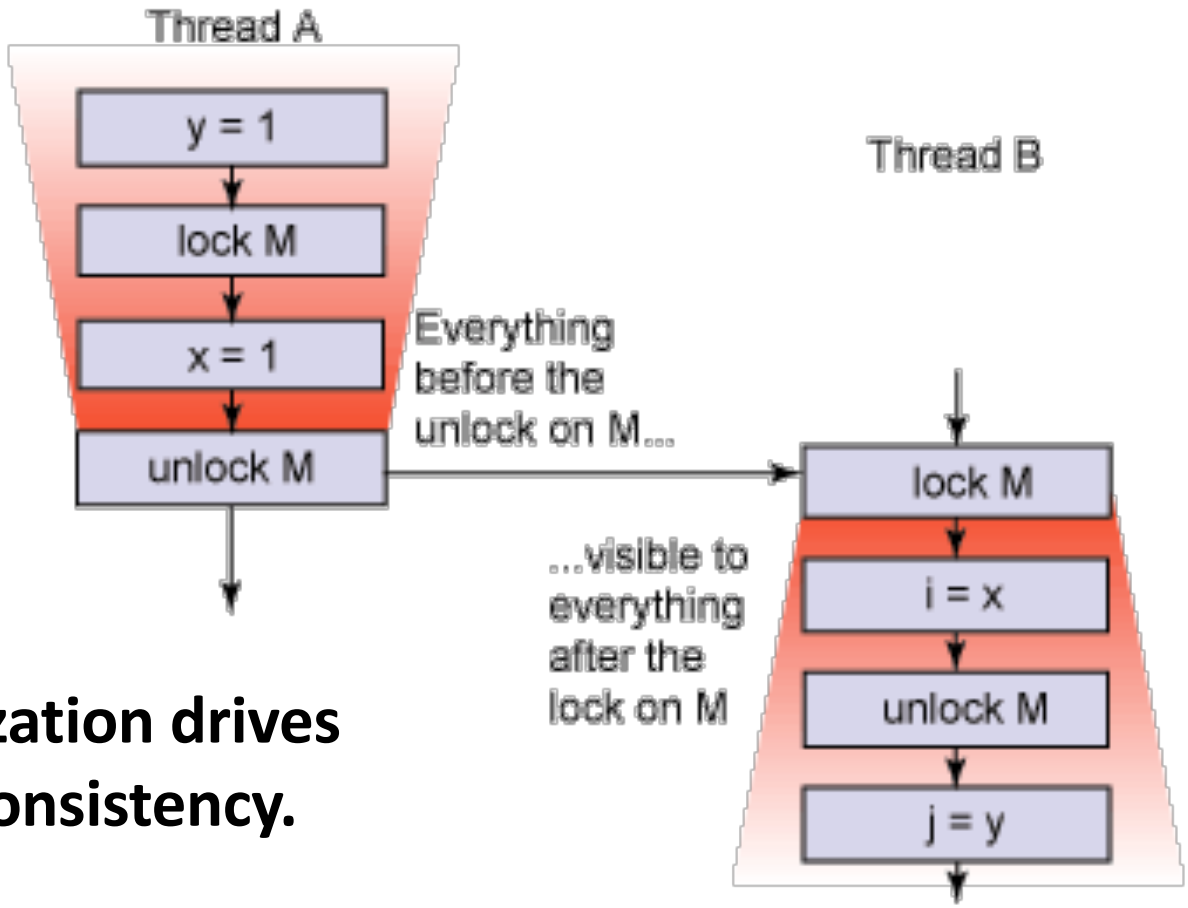
APRIL 2016 VOL. 59 NO. 4 COMMUNICATIONS OF THE ACM

Happened-before is a partial order

- **Partial order**: some events are **concurrent**: no ordering is defined by →.
- Cycles "cannot happen": event e1 cannot occur both before and after e2.
- Therefore, this program ~~can~~ ~~must~~ **should** not print "Hello World".
- (But it might on a modern system if you don't use your locks.)

Thread 1	**Thread 2**
`A = 1`	`B = 1`
`if (B == 0)`	`if (A == 0)`
`print "Hello";`	`print "World";`

Multicore memory systems today



Synchronization drives memory consistency.

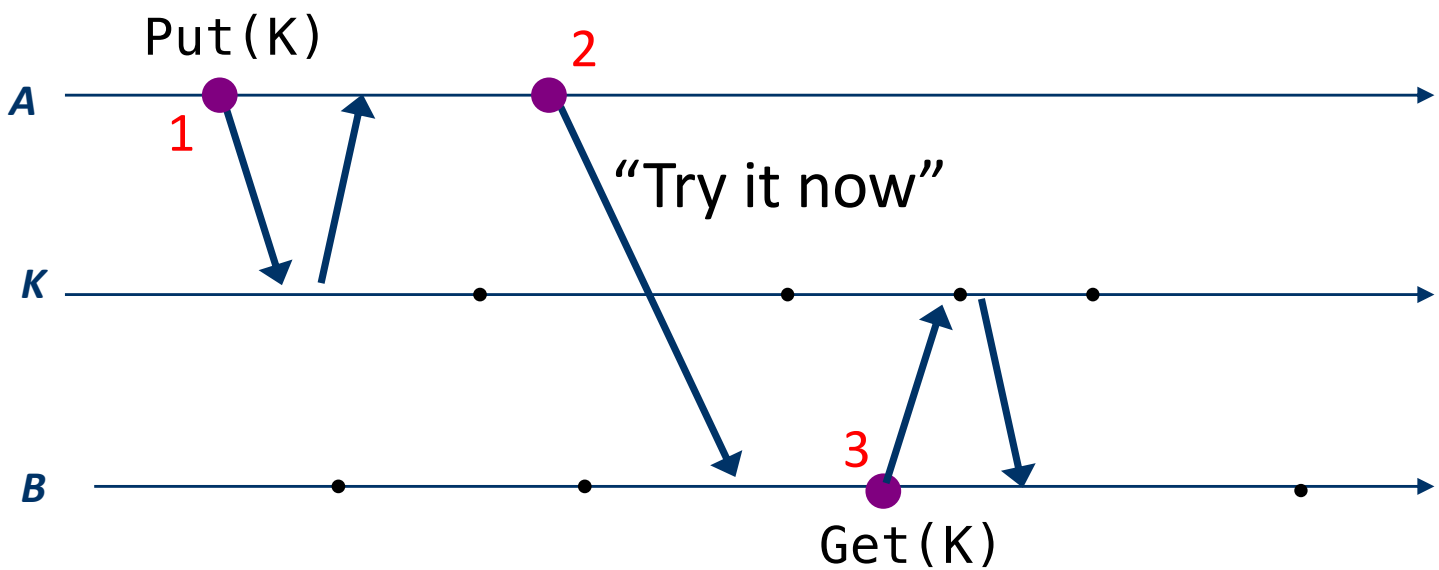
Logical clocks

1. Each node maintains a local **logical clock** value LC.
2. Events/messages/updates are timestamped with the current LC value at the generating node.
   - Increment LC on each new event: LC = LC + 1
   - LC is monotonically increasing
3. Attach the sender's current clock value LC_s to each message ("Piggyback").
4. Receiver advances its LC clock:

if $LC_s > LC$
then $LC = LC_s + 1$

Logical clocks and the clock condition

- *LC* ordering respects potential causality.
- **Clock Condition**: if $e_1 \rightarrow e_2$ then that $LC(e_1) < LC(e_2)$

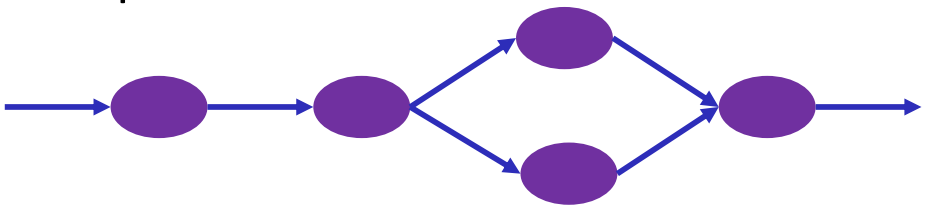


Causal consistency

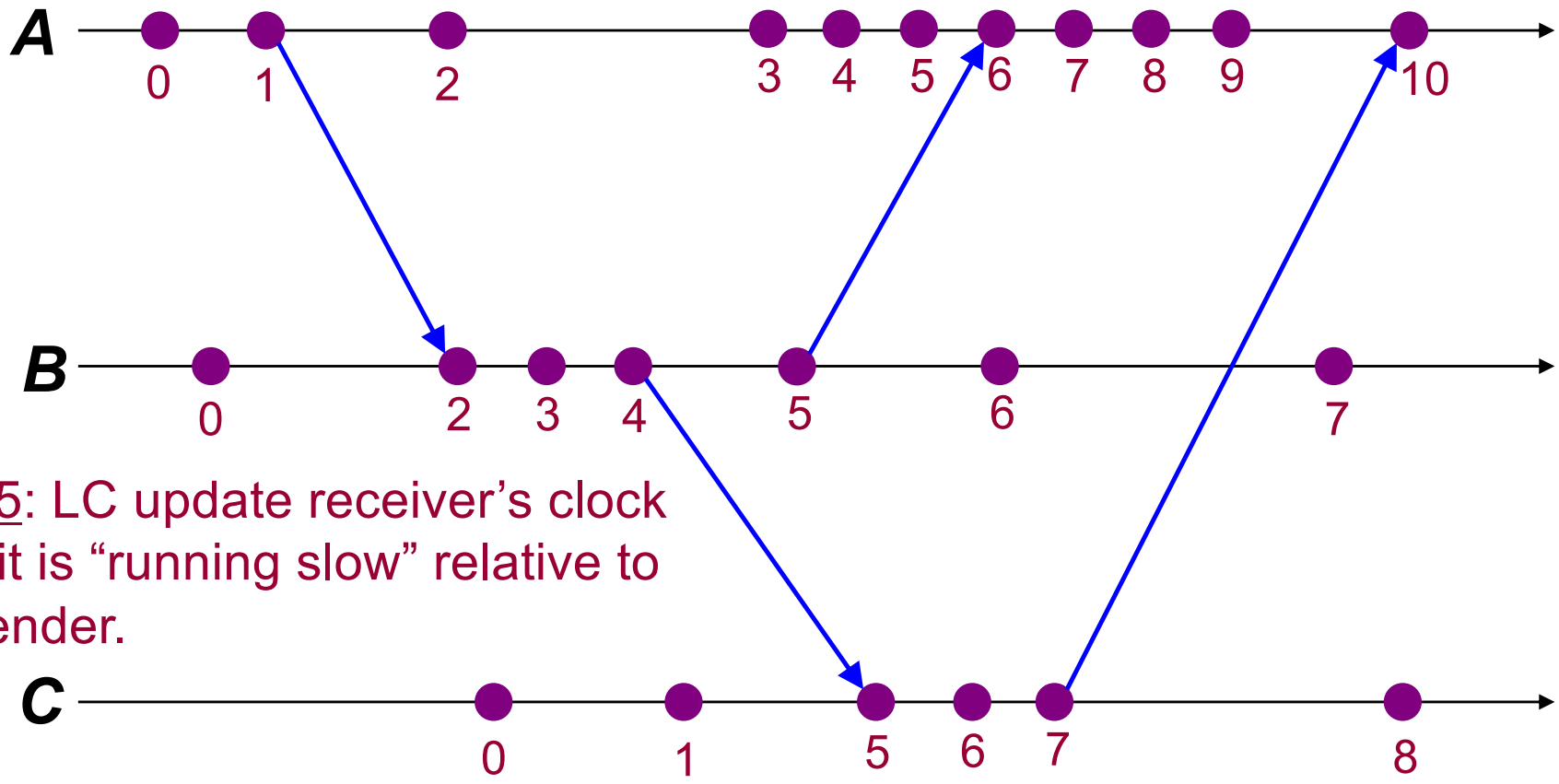
- Causal consistency assures that programs see events in a happened-before order: a causal order.
  - Note: there are many such orders.
- We want causal orderings: they capture an essential aspect of what we mean by "consistent".
  - E.g., modern memory systems and languages assure causal orders, at least for correct programs.
- Causal consistency is *weaker* than linearizabiity.
  - "Weaker" means the set of "legal" orderings is bigger (superset).
- Raw eventual consistency (e.g., Dynamo) does not guarantee causal ordering: nodes may observe a **causal inconsistency.**

Detecting concurrent events

- How to detect if events are **concurrent**?
- **Logical clocks don't help.**
- Specifically, the clock condition does not help, because its converse does not hold:
  - *$LC(e_1) < LC(e_2)$ does not imply $e_1 \rightarrow e_2$*
- If *A* observed all events in *B*, then $LC_A > LC_B$
- But just knowing that $LC_A > LC_B$ **does not make it so**.
  - "False positives"
  - "We're gonna need a bigger timestamp!"



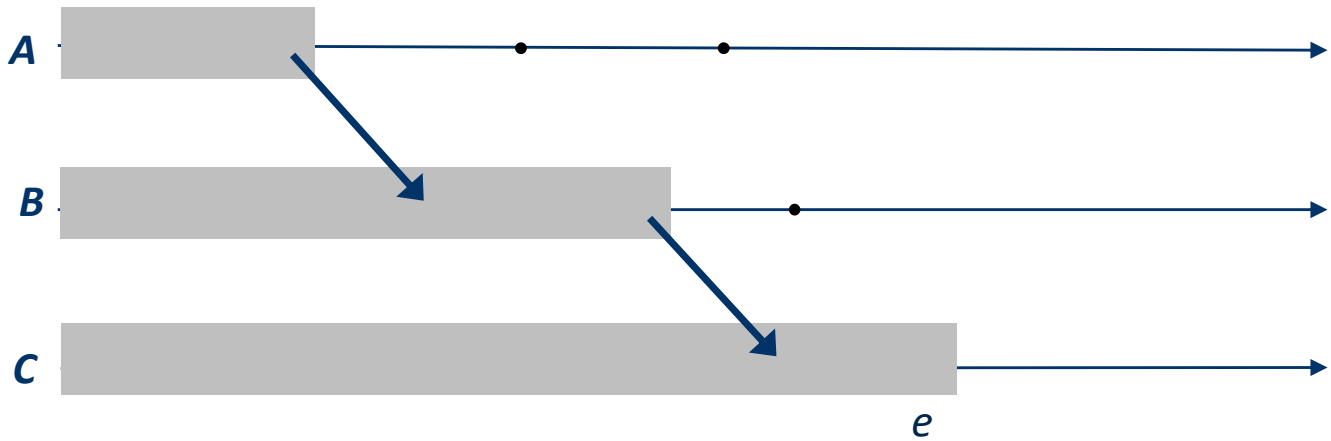
A6-A10: receiver's clock is unaffected because it is "running fast" relative to sender.



C5: LC update receiver's clock if it is "running slow" relative to sender.

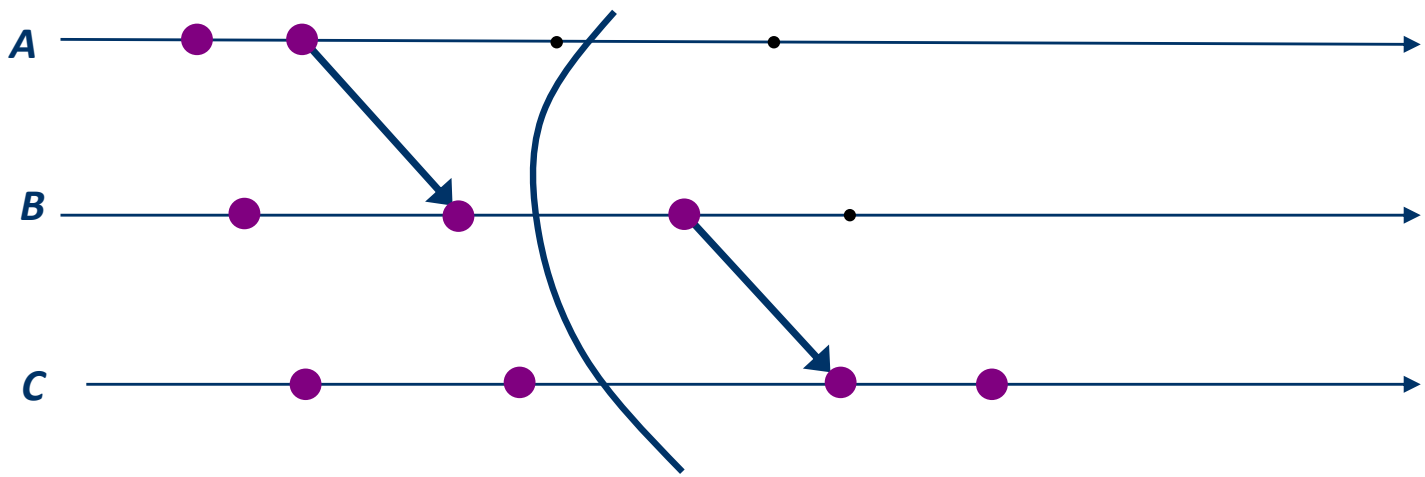
Events in a distributed system

- Sequential history of events in each process/node/thread.
- A local state of a node is defined by a prefix of its **local history**: an event and its predecessors in program order.
- Communication ("touch") generates causal relationships.
- Shaded prefixes comprise the **causal history** of event *e*.



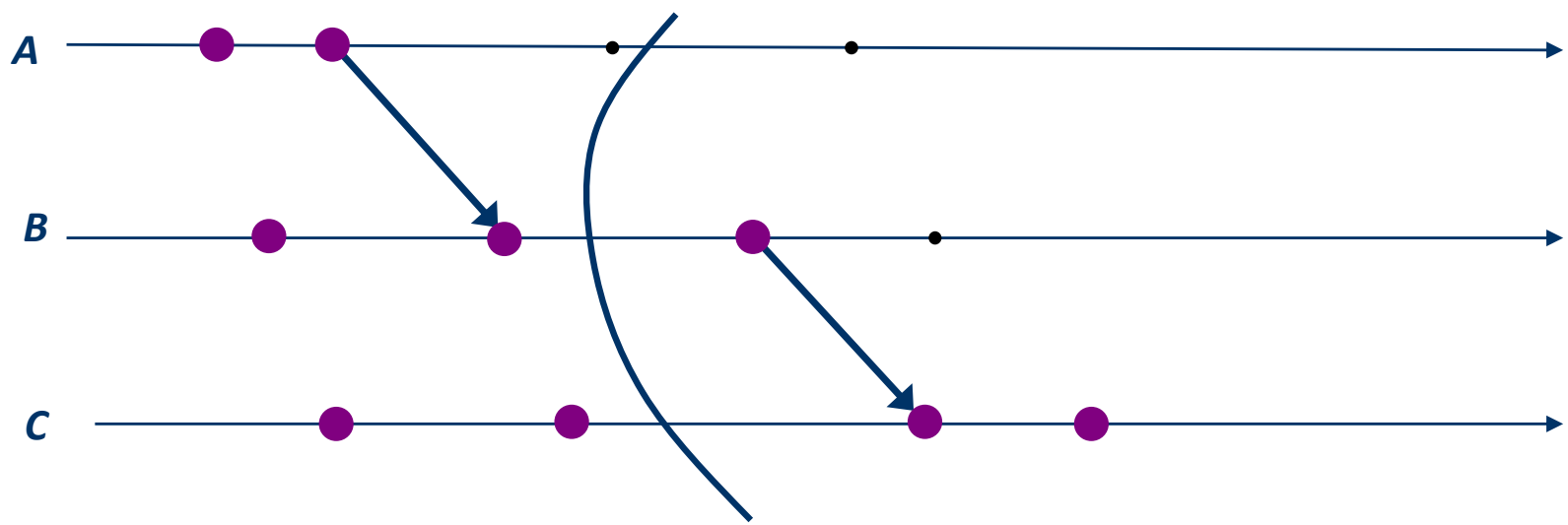
States of a distributed system: cuts

- A **cut** is a subset of the global history containing a local state (local history prefix) for each node.
- Each cut corresponds to a **global state** of the system.
- A **vector clock** is an array of logical clock values for each node, where some cut intersects the node's timeline.



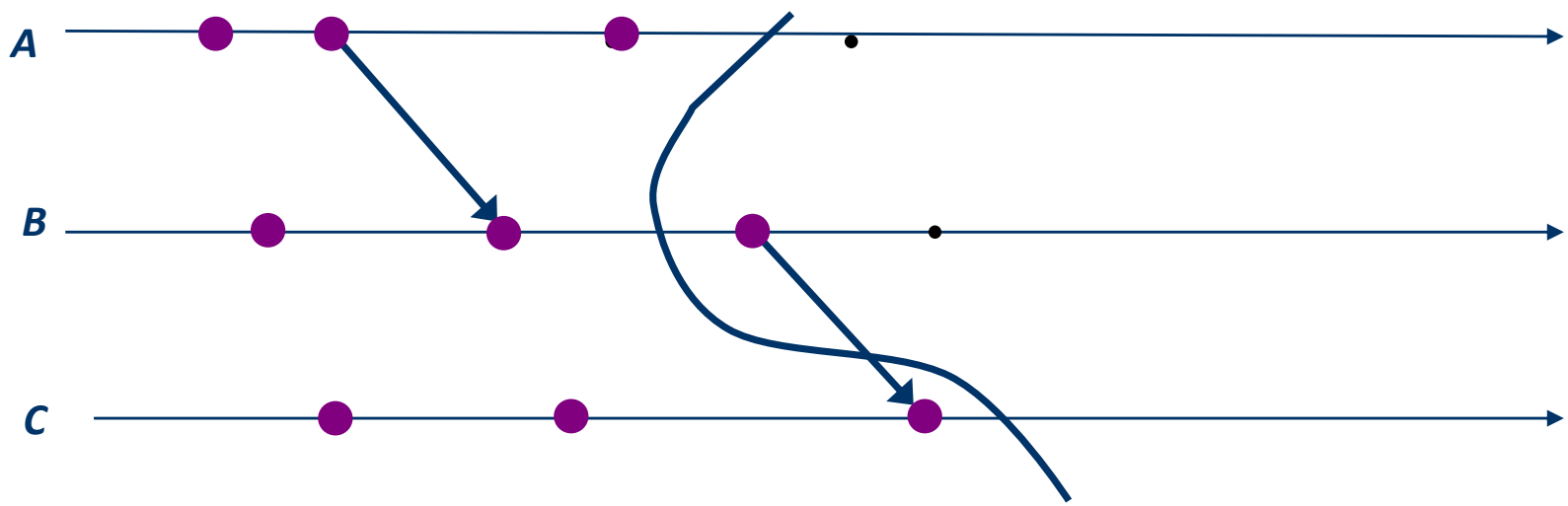
Consistent cuts

- A cut C is a **consistent** cut iff the following condition is true: for every event $e \in C$, if $\exists e' \rightarrow e$ then $e' \in C$.
- Equivalently, a **consistent cut** includes the complete causal history of any event in the cut: *left-closed* under *happened-before*.
- C corresponds to a **consistent global state**.



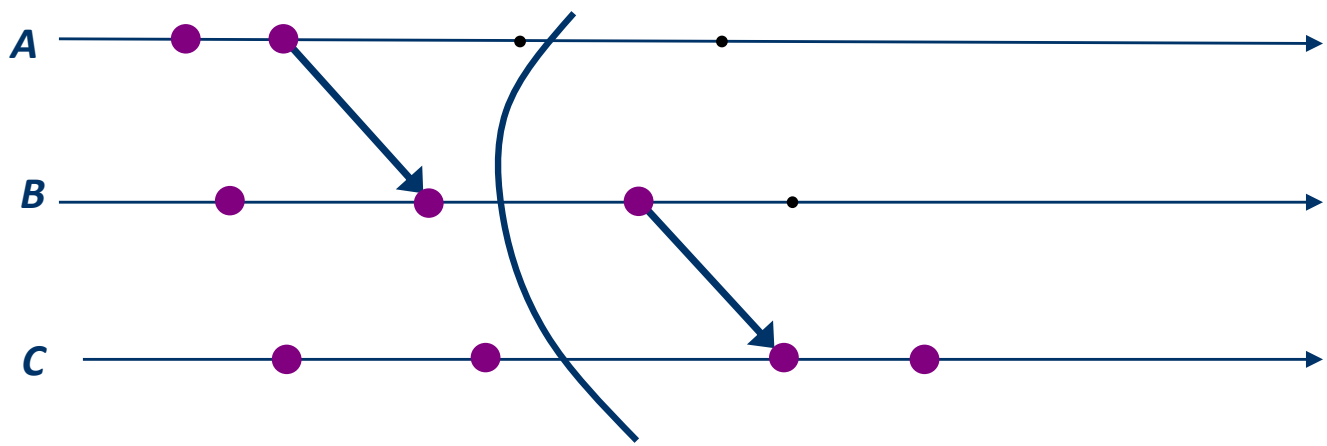
Inconsistency

- An observer of *e* may perceive an inconsistency if it does not know all events in *e*'s causal history.
- A cut C is **inconsistent** cut if it misses a causal predecessor of some event in the cut: C corresponds to an **inconsistent global state**.



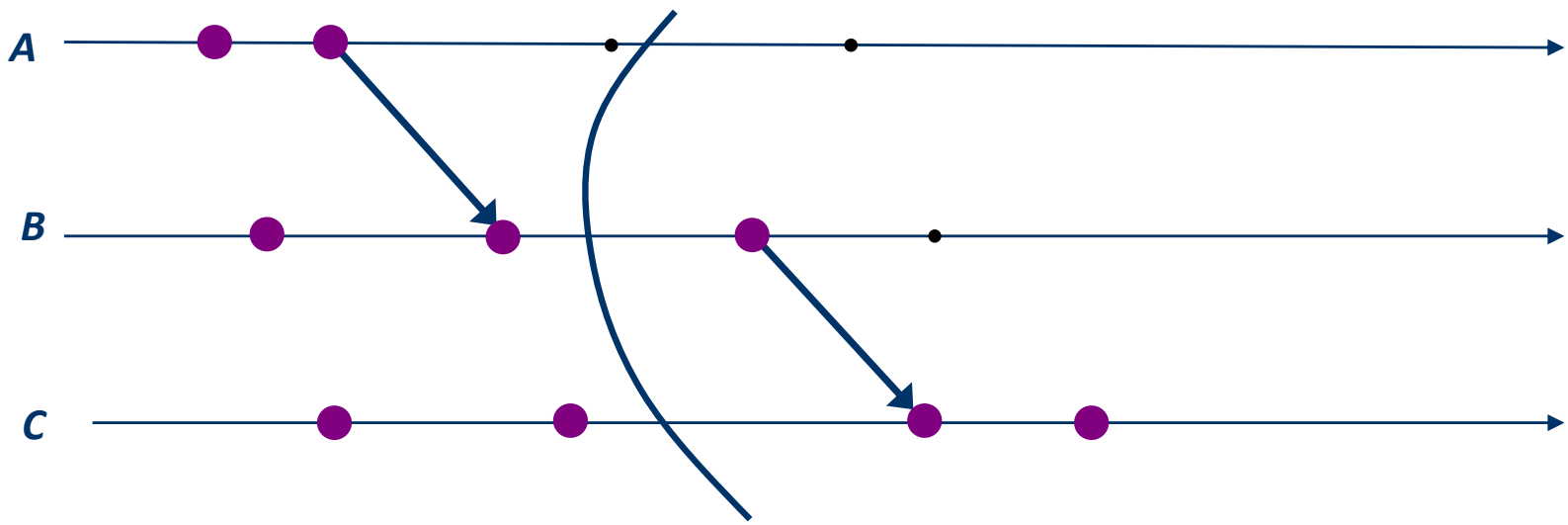
Vector clocks

- **Vector clocks** (also called **vector timestamps**) induce an order that *exactly* reflects causality and concurrency.
- Each of *N* nodes keeps a **vector clock** *TS[N]* and its logical clock *LC*.
  - *TS[j]* at node *i* is the most recent value of node *j*'s logical clock that *i* observed or "saw" or "heard about".
  - E.g, $TS_i[i] = LC_i$



A vector clock corresponds to a cut

- The **vector timestamp** of a cut is a vector of logical clock values: the local logical clock of the last event in each local history prefix of the cut.
- Vector timestamps/clocks capture observed global states.

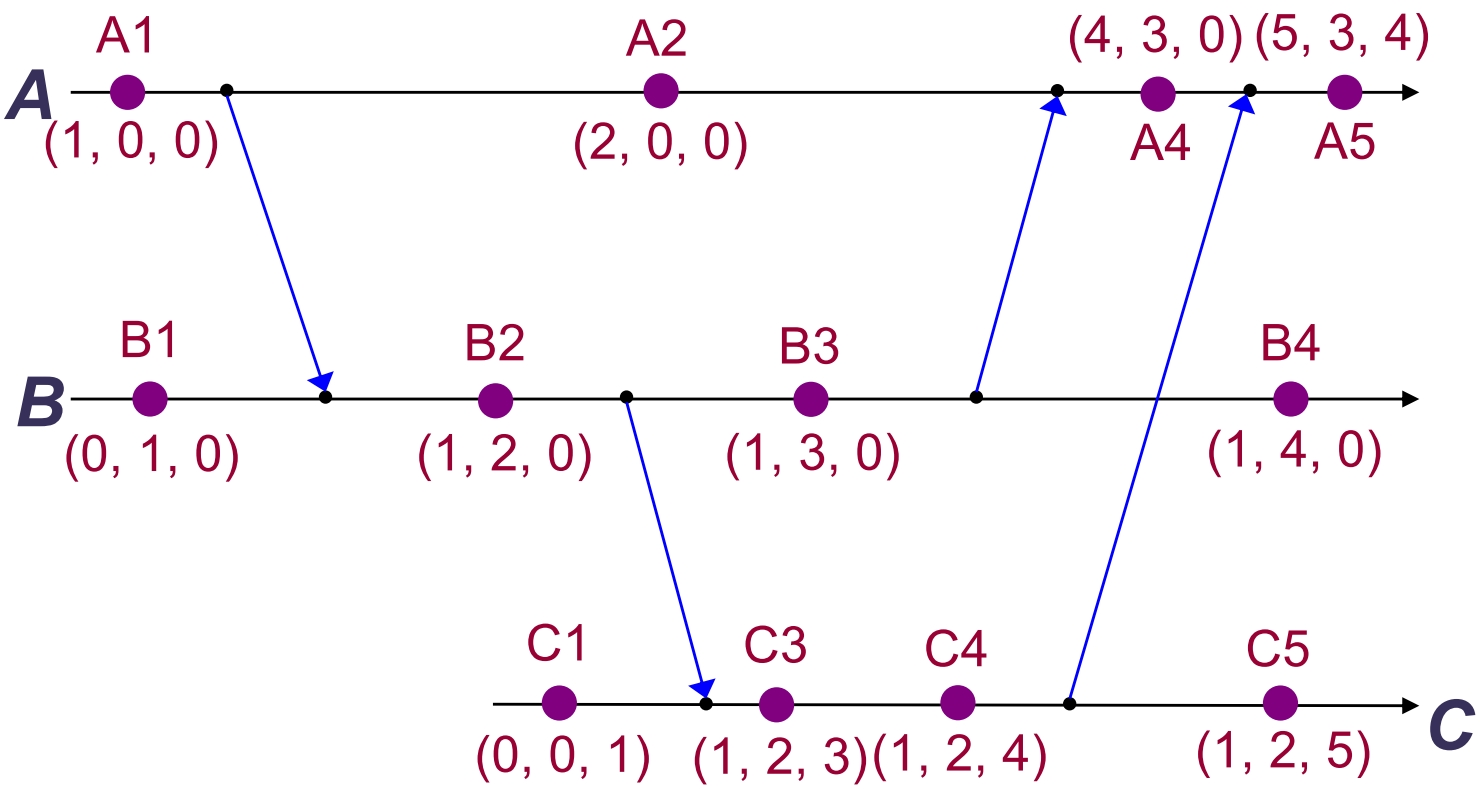


Updating a vector clock (join)

- A node *r* observing an event (e.g., receiving a message) from node *s* may learn its causal history.
- Merge (union) sender's causal history with its own.
- Set the TS_r to the **pairwise maximum** of TS_s and TS_r (**join**).
  - $TS_r[i] = \text{MAX}(TS_r[i], TS_s[i])$ for every i

- The **join** of two cuts is the minimal cut containing both causal histories.

Vector clocks: example

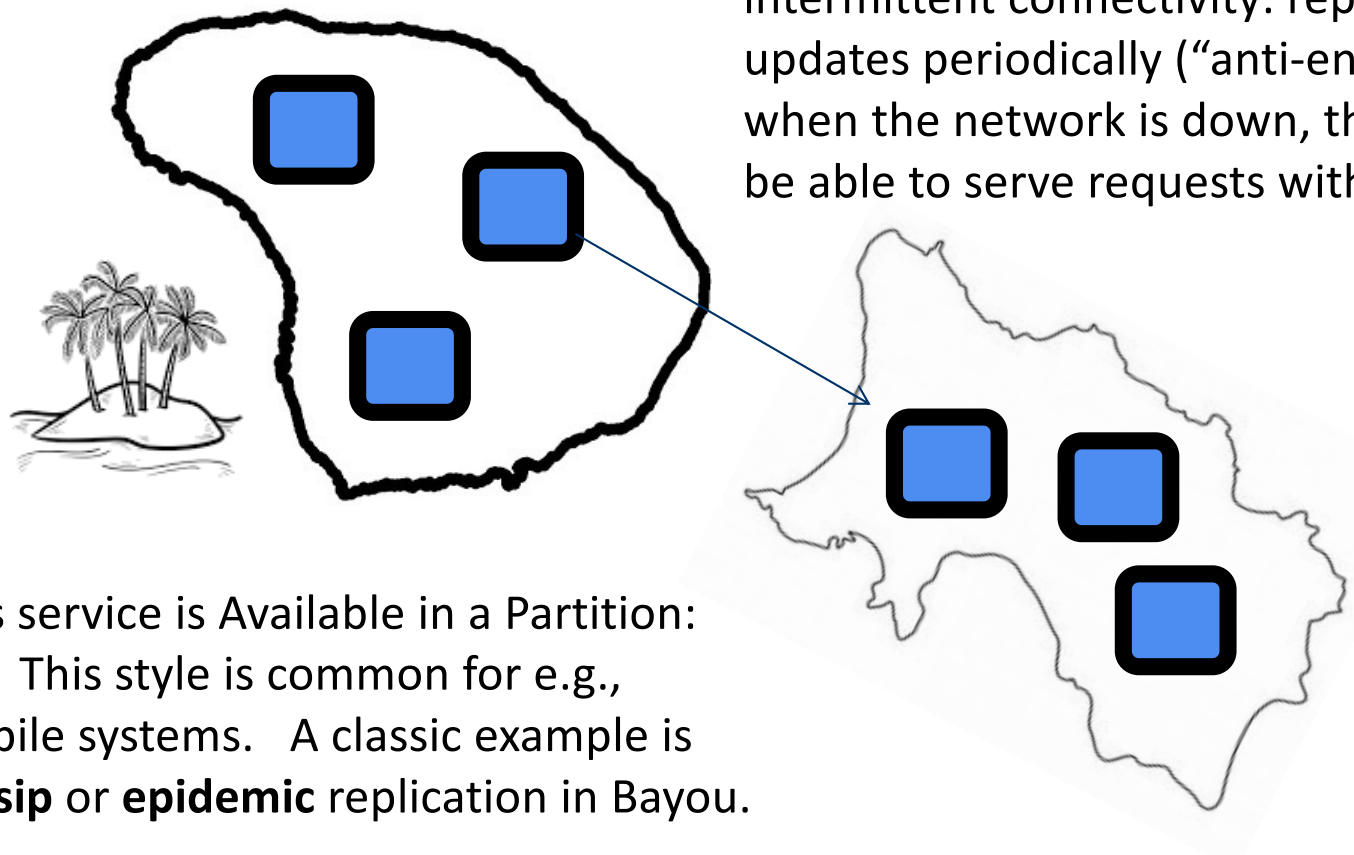


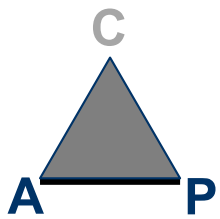
Vector clocks and causality

- *$e_1 \rightarrow e_2$ if and only if $TS(e_2)$* ***dominates*** *$TS(e_1)$*
  - $e_1 \rightarrow e_2$ iff $TS(e_1)[i] <= TS(e_2)[i]$ for each i
  - "Every event or update visible when e_1 occurred was also visible when e_2 occurred".
- We can **compare** vector clocks to determine if two events are **concurrent**: neither *happened-before* the other.
  - If $e_1 \rightarrow e_2$ then $LC(e_1) < LC(e_2)$ and $TS(e_2)$ dominates $TS(e_1)$.
  - If $TS(e_2)$ does not dominate $TS(e_1)$ then it is not true that $e_1 \rightarrow e_2$.
  - **Iff neither vector dominates the other, they are concurrent.**

Extreme AP: replicas on island travel

Consider a replicated service S with intermittent connectivity: replicas exchange updates periodically ("anti-entropy"). If S is up when the network is down, then a replica must be able to serve requests without coordinating.



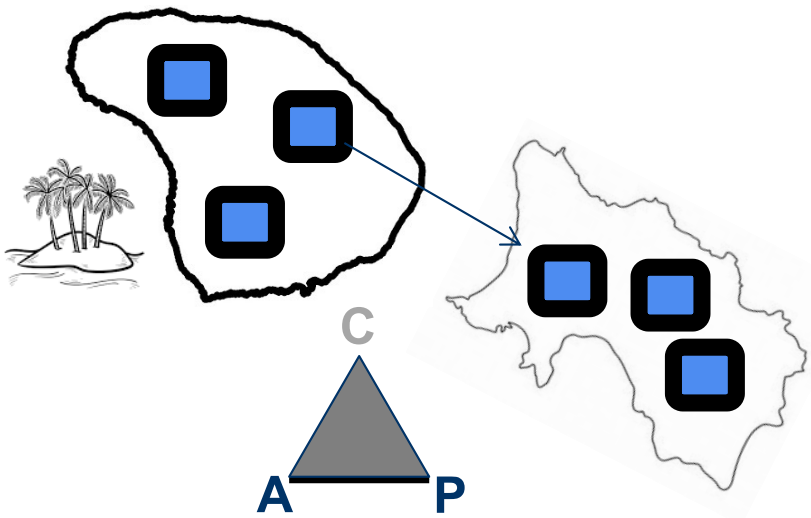


This service is Available in a Partition: AP. This style is common for e.g., mobile systems. A classic example is **gossip** or **epidemic** replication in Bayou.

Anti-entropy / epidemic replication

Each replica periodically “reaches out and touches” a selected “susceptible” peer and “infects” it with ops/updates/gossip.
Eventually every replica learns of every update: eventual consistency.
Classic epidemic systems preserve causality by “flooding” updates.

Epidemic replication: cartoonishly oversimplified



1. Tag each update/op with (origin, LC) and apply immediately to the local replica.
2. When a replica "touches" another, transfer ops not known to the other. (**anti-entropy exchange** "infects" with updates)
3. Each node maintains a vector clock that captures all ops it "has seen".
4. On a "touch", the difference of the vector clocks gives exactly the ops to transfer.
5. Order (or reorder) updates in LC order, and use the origin to break ties.
6. Application must handle rollback/reorder and deterministic merge.

All replicas **converge** to the same timeline as updates propagate. For **tentative** intermediate states: every order in every replica is a causal order.

Dynamo: write conflict scenario

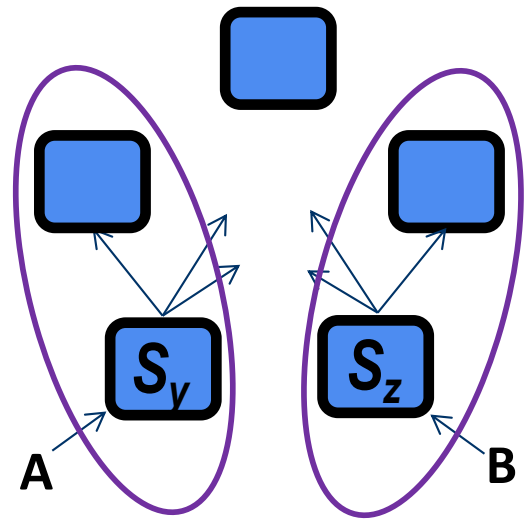
- **Suppose** clients A and B both put on key K "at the same time", and:
  - W knob is set low: writes commit with less than a majority.
  - A selects server S_x to coordinate and B selects a different server S_z.
  - S_x commits A's put(K) without observing B's put(K).
  - S_y commits B's put(K) without observing A's put(K).
- These puts are **concurrent** and **conflicting**.

  No server observes both puts before both commit.

  → No server can impose a lock.

  → No clear "winner"—which put is first or last.

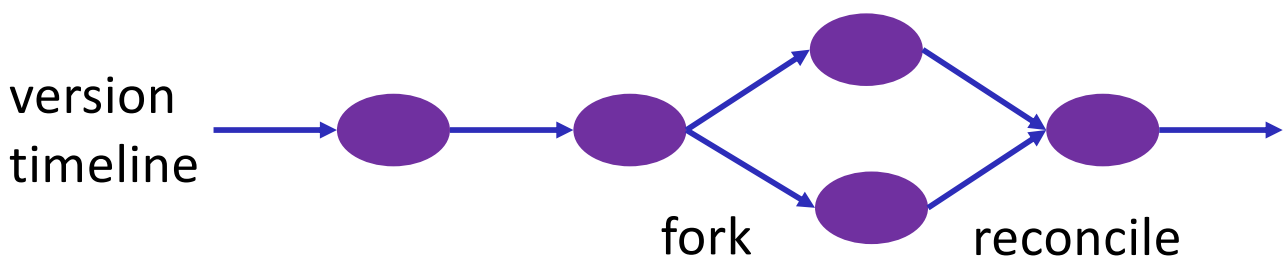
  → **Multiple versions for K**



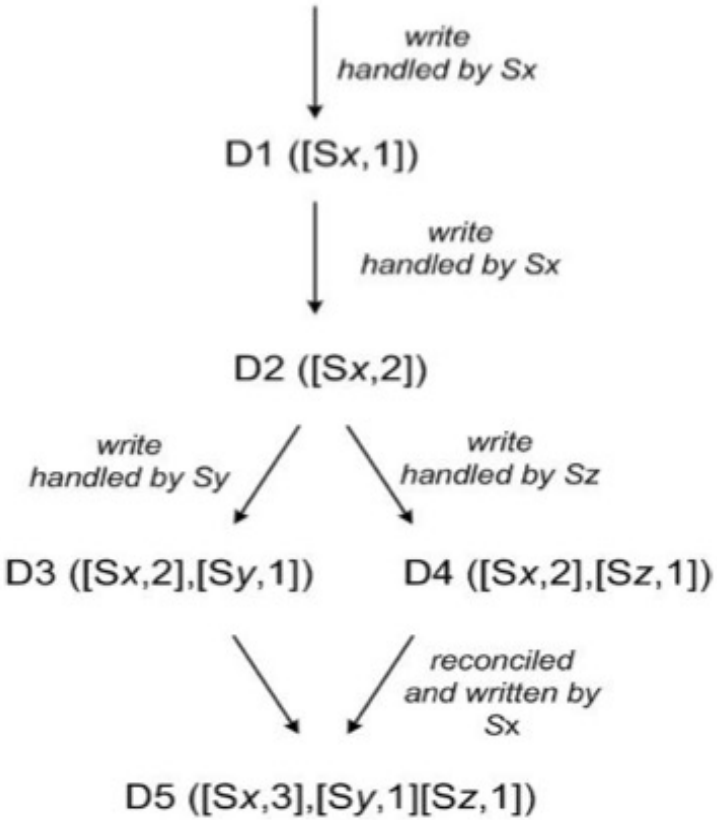
N=5; W=2

Reconciliation (version merge)

- Version conflicts → a fork in the version history for at least one key K.
- On the next get(K), the client C discovers the conflict.
- C must compare versions and **merge** the values in a way that works for the application data and its type.
  - E.g., revert and reorder the operations
  - Merge must be deterministic so replicas converge
- C must put(K) with a single merged version/value, ordered **after** the inputs to the merge.



Dynamo timestamps are vector clocks



- A Dynamo vector clock is a list of ([node,count] pairs.
- Each node is a coordinator.
- The count is the number of writes (versions) that the coordinator has handled for the object.
  - So, not really a logical clock—it reflects only "program order"—but close enough.
- **Compare** vectors to detect forks.
- **Join** vector clocks to timestamp the merged state.

In conclusion

- Causality matters: captures what we mean by consistency.
  - Concurrent updates can be seen in either order.
- Logical clocks capture causality but not concurrency.
- Vector clocks capture causality and concurrency.
- These clocks have many many uses in distributed systems.